

# AI Model Gateway Customer Guide

A simple reference for customers who want a multi-model AI platform direction, but may not have a large technical team yet.

|           |                                                           |
|-----------|-----------------------------------------------------------|
| Audience  | Business and product teams with basic technical awareness |
| Prototype | Local AI Model Gateway with Qwen / DashScope first        |
| Main idea | One customer API, many model providers behind the scenes  |
| Demo mode | Mock mode first, live Qwen mode only when needed          |

## Executive Summary

An AI Model Gateway lets customers call one simple API while the platform manages model routing, provider differences, usage control, and future fallback logic. The first prototype uses Alibaba Cloud Model Studio / Qwen because it is available now and keeps testing cost low.

- Start with Qwen and mock mode before adding more paid providers.
- Use an OpenAI-compatible API so customers can understand the integration quickly.
- Show routing visually: smart-fast is a customer-facing name that maps to qwen-plus.
- Keep enterprise API Gateway features optional until customer demand is clear.

## Layered Architecture

| Layer                    | What It Means                                   | Example                         |
|--------------------------|-------------------------------------------------|---------------------------------|
| 1. Customer Systems      | The customer's app, backend, or agent           | Website, chatbot, internal tool |
| 2. One Simple API        | One standard API format for customers           | /v1/chat/completions            |
| 3. Gateway Entry Control | Controls who can call and how much they can use | API keys, limits, logs          |
| 4. Model Control         | Chooses the public model and route              | smart-fast -> qwen-plus         |
| 5. Provider Adapters     | Translates request and response formats         | DashScope adapter               |
| 6. Model Providers       | The real upstream AI model provider             | Alibaba Cloud Qwen              |

The dashboard also shows this as a visual architecture map, so non-technical customers can follow the layers from customer systems to model providers.

## How Route Simulation Works

The dashboard explains routing without spending money. In mock mode, the gateway reads the requested model, checks the registry, maps the public model to a real upstream model, and returns a simulated response with a route trace.

```
Customer request: model = smart-fast
Registry lookup: smart-fast maps to qwen-plus
Provider adapter: prepare DashScope-compatible request
```

Mock response: show the route trace without calling Alibaba Cloud

## What The Current Prototype Includes

- Visual dashboard at the gateway root URL.
- OpenAPI contract at /openapi.json for customer technical handoff.
- Postman collection at /postman\_collection.json for click-through demos.
- Demo bundle manifest at /v1/gateway/demo-bundle for customer presentation flow.
- Customer integration guide at /v1/gateway/integration-guide with safe code examples.
- OpenAI-compatible /v1/chat/completions endpoint.
- Customer API key authentication with Bearer token.
- Basic in-memory request limit.
- Model registry in model\_registry.json.
- SQLite request and usage records for restart-safe demo data.
- Provider, customer, model, and request summary endpoints.
- Operational alerts endpoint with simple next steps.
- Incident playbook endpoint with support scenarios and customer-safe wording.
- Support policy endpoint for prototype, pilot, and production support stages.
- Pilot checklist endpoint for before, during, and after a customer trial.
- Executive brief endpoint for non-technical customer stakeholders.
- Roadmap endpoint for prototype, pilot, production hardening, and multi-provider expansion.
- Decision guide endpoint for API Gateway, managed AI Gateway, custom Model Gateway, and OpenRouter-like options.
- FAQ endpoint for common customer questions and objections.
- Demo script endpoint for a 15 minute customer walkthrough.
- Access matrix endpoint for customer and model permissions.
- Provider health endpoint for mock-ready, live-ready, degraded, and not-ready states.
- Provider create, update, and disable actions for provider lifecycle demos.
- Provider contract matrix for adapter differences across Qwen, OpenAI-compatible APIs, Claude, and planned providers.
- Model catalog endpoint for provider status, fallback chain, usage, and pricing metadata.
- Model route create, update, and disable actions for route lifecycle demos.
- Route preview endpoint to dry-run model access, budget, provider readiness, and fallback order.
- Cost estimate endpoint to dry-run token estimate, estimated cost, and budget impact.
- Customer reports endpoint for request count, errors, token usage, and budget state.
- Request activity endpoint with simple filters for troubleshooting.
- Request detail lookup using gateway.request\_id returned in chat responses.
- Config check endpoint for demo keys and missing production settings.
- Production readiness endpoint for plain-English go-live gaps.

- Structured route decision summary for support explanations.
- Local safety preview for obvious sensitive data.
- Customer key issue preview for safe onboarding demos.
- Customer key create, rotate, and disable actions for lifecycle demos.
- Audit events for customer key lifecycle actions.
- Customer default policy for automatic routing presets.
- Invoice preview with JSON and CSV output.
- Capability routing control for streaming and tools.
- Request-level fallback controls for routing demos.
- Request-level provider allow-list for routing control demos.
- Request-level route strategy for registry, lowest cost, fastest, and healthiest routing.
- Named policy presets for common routing controls.
- Mock OpenAI-style tool call response for agent demos.
- Simple customer plans with token and cost budgets.
- Bring Your Own Key mapping through environment variables.
- Provider adapter scaffolds for OpenAI-compatible APIs and Claude-style APIs.
- Automated mock regression test that does not spend provider credits.
- Mock mode for demos and planning.
- Live Qwen mode when DASHSCOPE\_API\_KEY is available.

## Main Technical Difficulties

| Difficulty                  | Why It Matters                                                             | Prototype Status                                    |
|-----------------------------|----------------------------------------------------------------------------|-----------------------------------------------------|
| Provider format differences | Each provider may use different request and response details.              | Handled first for DashScope-compatible chat         |
| API handoff                 | Customer technical teams need a clear API contract.                        | OpenAPI contract endpoint                           |
| Demo handoff                | Customer technical teams may want to click through requests.               | Postman collection endpoint                         |
| Demo flow                   | Business and technical users need to know what to look at.                 | Demo bundle manifest endpoint                       |
| Customer integration        | A customer technical team needs safe examples for their own integrations.  | Customer integration guide endpoint                 |
| Streaming                   | Chat UIs and agents often need incremental tokens.                         | Mock streaming included; live provider differences  |
| Tool calling                | OpenAI, Claude, and Qwen may differ in tool format.                        | Mock tool_calls plus basic normalization            |
| Token usage                 | Billing and quota require reliable usage data.                             | Stored in SQLite                                    |
| Operational alerts          | Non-technical users need a clear action list.                              | Alerts with severity, area, and next step           |
| Incident response           | Support teams need a shared script when requests fail.                     | Playbook with signals, steps, and customer wording  |
| Support policy              | Customers need to know what support is promised at each stage.             | Prototype, pilot, and production support guide      |
| Pilot planning              | A customer trial needs scope, roles, success criteria, and exit strategy.  | Pilot checklist endpoint                            |
| Executive communication     | Non-technical stakeholders need a short business summary.                  | Executive brief endpoint                            |
| Roadmap planning            | Customers need to see the path from demo to production.                    | Roadmap endpoint                                    |
| Build or buy decision       | Customers need to compare gateway options clearly.                         | Decision guide endpoint                             |
| Customer objections         | Sales and support need consistent answers to common questions.             | FQA endpoint                                        |
| Demo delivery               | A presenter needs a simple meeting flow, talk track, and likely questions. | Demo script endpoint                                |
| Production readiness        | Customers need to understand why demo-ready is not production-ready.       | Plain-English readiness report                      |
| Access control              | Each customer may be allowed to use different models.                      | Access matrix by customer and model                 |
| Provider health             | Customers need to know whether a provider can serve traffic.               | Readiness view based on config and recent traffic   |
| Provider lifecycle          | Teams need to add and stop providers safely.                               | Local JSON-backed create, update, and disable a     |
| Provider contracts          | Teams need to see why each provider needs adapter tests.                   | Contract matrix for provider differences            |
| Model catalog               | Customers need to understand public model names and routes.                | Catalog with provider status, fallback chain, usage |
| Model route lifecycle       | Teams need to change public model routes safely.                           | Local JSON-backed create, update, and disable a     |
| Route preview               | Sales and support need to explain a route before spending credits.         | Credits endpoint and dashboard preview              |
| Route decision summary      | Customers may ask why a route was chosen.                                  | Plain-language summary and reasons                  |
| Safety preview              | Customers may worry about secrets in prompts.                              | Local preview for obvious emails, phone numbers     |

|                          |                                                                |                                                        |
|--------------------------|----------------------------------------------------------------|--------------------------------------------------------|
| Provider routing control | Some customers may want only approved providers for a request. | gateway_allowed_providers filters route candidates     |
| Route strategy           | Customers may want cost, latency, or health-aware routing.     | Request-level route strategy reorders candidates       |
| Policy presets           | Non-technical users need simple policy names.                  | gateway_policy applies named routing presets           |
| Customer default policy  | Customers may not want to send routing controls every time.    | Customer config can set a default policy               |
| Capability routing       | Requests may need streaming, tools, or other abilities.        | gateway_required_capabilities filters route candidates |
| Cost estimate            | Customers need budget planning before live calls.              | Dry-run estimate for tokens, cost, and budget impact   |
| Customer onboarding      | New customers need keys, limits, and model access.             | Key issue preview creates a safe config snippet        |
| Key lifecycle            | Customers need key rotation and disable workflows.             | Local JSON-backed create, rotate, and disable actions  |
| Audit trail              | Teams need to know who changed customer access.                | Audit events for customer lifecycle actions            |
| Customer self-service    | Customers need to see their own access, usage, and budget.     | Customer self view with no provider secret exposure    |
| Invoice preview          | Customers need a simple billing story.                         | Estimated invoice preview with CSV export              |
| Customer reporting       | Customers need a simple usage and budget story.                | Per-customer report endpoint and dashboard cards       |
| Request troubleshooting  | Support teams need to see what happened to a request.          | Filtered request activity feed                         |
| Request detail           | Support teams need one-request lookup.                         | gateway.request_id and request detail endpoint         |
| Fallback                 | Retrying another model needs clear business rules.             | Registry fallback plus request-level controls          |
| Customer key management  | Each customer needs limits, logs, and access control.          | Minimum version plus BYOK mapping                      |
| Cost control             | Different models have different prices and limits.             | Simple token and cost budgets included                 |

## Why The Test Script Matters

The repository includes `test_gateway.py`. It starts the gateway in mock mode and checks API keys, model listing, route tracing, fallback, model access rules, token budget blocking, SQLite logs, and status output without leaking provider keys.

## API Contract

`/openapi.json` exposes an OpenAPI contract for the prototype. It describes the customer API, admin control-plane API, bearer key security schemes, and the main endpoint groups. `/postman_collection.json` exposes a ready-to-import Postman collection with `base_url`, `gateway_api_key`, and `admin_api_key` variables. `/v1/gateway/demo-bundle` lists the dashboard, customer self view, OpenAPI contract, Postman collection, PDF guide, recommended demo order, safe curl examples, and production notes. The dashboard also has a Customer handoff package section that links the main artifacts in one place and an Integration command starter with local curl commands. `/v1/gateway/integration-guide` lets a customer use their own key to see allowed models, a recommended first model, curl, Python, JavaScript, streaming examples, and a go-live checklist without exposing provider secrets. `/v1/gateway/sdk-starter` adds a `.env` template, starter Python and JavaScript files, first-run commands, common error fixes, and a handoff checklist. These help a business team understand the story first and help a customer technical team import or click through the prototype quickly.

## Admin Summary Endpoints

The prototype exposes local admin JSON views for provider status, operational alerts, access matrix, provider health, model catalog, route preview, customer reports, request activity, request detail lookup, usage by customer, usage by model, and request summaries. These make the control layer easier to explain. The local demo uses a separate admin key. In production, this key should be changed and protected.

## Operational Alerts

`/v1/gateway/alerts` combines config warnings, provider readiness, customer budget state, and recent request errors. Each alert includes severity, area, message, and a simple next step. This helps non-technical users understand what needs attention before a live demo.

## Incident Playbook

`/v1/gateway/incident-playbook` explains common failure scenarios in simple English. It covers provider readiness, recent request errors, customer budget blocks, demo key or secret risks, and provider contract gaps. Each scenario includes triggers, signals to check, current evidence, operator steps, and customer-safe wording. It is not a legal SLA, but it helps sales, support, and technical teams explain issues consistently.

## Support Policy

`/v1/gateway/support-policy` explains prototype, technical pilot, and production target support expectations. It defines P1, P2, and P3 severity, first checks, escalation path, and customer-safe wording. It clearly says the current prototype is not a legal production SLA. This helps customers understand what can be promised now and what needs a real contract later.

## Pilot Checklist

`/v1/gateway/pilot-checklist` helps prepare a small customer trial. It covers recommended scope, roles, before-pilot checks, during-pilot checks, after-pilot review, success criteria, and the stop, extend, or productionize decision. It helps keep the pilot small, honest, and measurable.

## Onboarding Plan

`/v1/gateway/onboarding-plan` turns the demo into a simple customer pilot path. It explains Day 0 customer alignment, Day 1 safe access setup, Day 2 mock technical test, Day 3 route and provider review, Day 4 usage, cost, and support review, and Day 5 pilot decision. Each step has an owner, actions, evidence, and an exit check, so non-technical customers can understand what happens after the demo.

## Executive Brief

`/v1/gateway/executive-brief` gives a short business summary for non-technical stakeholders. It explains the one-sentence value, why the gateway matters, what the demo proves, what is not production-ready yet, the recommended customer story, pilot recommendation, support and readiness position, and next step.

## Roadmap Endpoint

`/v1/gateway/roadmap` explains the path from prototype to production. It includes prototype explanation, technical pilot, production hardening, and multi-provider expansion. Each phase has a goal, deliverables, exit criteria, and main risks. This helps customers understand next steps without jumping too quickly into production promises.

## Decision Guide

`/v1/gateway/decision-guide` compares normal API Gateway, managed AI Gateway, custom Model Gateway, and OpenRouter-like marketplace options. The dashboard also renders this as Gateway options comparison cards. It explains what each option is good at, what it does not solve, and when to choose it. This directly answers the early question: is an API Gateway enough?

## Customer FAQ

`/v1/gateway/faq` answers common customer questions about API Gateway, Qwen, OpenRouter, prompt and key exposure, cost control, provider failure, adding more providers, pilot next steps, and production readiness. It helps sales, support, and technical teams answer in the same simple language.

## Demo Script

`/v1/gateway/demo-script` gives a 15 minute customer walkthrough. The dashboard also shows this as Presenter mode. It tells the presenter what to say first, which endpoint to show, how to explain routing, how to answer common objections, and how to close with a small pilot. This helps non-technical customers understand the project without reading code.

## Customer Self View

`/v1/gateway/me` lets a customer use their own gateway key to see their own plan, allowed models, budget state, usage, recent requests, and invoice preview. It does not expose raw customer API keys or provider API keys. This is useful when the customer asks: what can I use, and how much have I used?

## Access Matrix

`/v1/gateway/access-matrix` shows which customers can use which public models, why access is allowed or blocked, the customer's budget state, and the provider readiness for each model. This helps explain customer-level API control.

## Provider Health

`/v1/gateway/provider-health` answers a simple question: can this provider serve traffic now? It checks enabled models, live key readiness, customer BYOK readiness, recent errors, and average latency. In mock mode, `ready_mock` means the provider can be explained without spending credits. It does not mean the live provider key is ready.

## Provider Lifecycle

The prototype can create, update, and disable providers. The dashboard has a Provider lifecycle panel for these demos. A provider defines the upstream API type, base URL, and API key environment variable name. Disabling a provider also disables active model routes that point to it, so the local registry stays valid. These admin actions update `model_registry.json`, reload the local runtime, and write audit events. The prototype stores the provider key environment variable name, not the provider secret value. Production should use a database, secret manager, approval workflow, readiness checks, and rollout controls.

## Provider Contract Matrix

`/v1/gateway/provider-contracts` explains why an AI Model Gateway is more than a normal API Gateway. It compares OpenAI-compatible providers, Alibaba Cloud Model Studio compatible mode, Anthropic Claude, and planned Xiaomi

or other local model providers. It shows auth, endpoint path, request shape, response shape, streaming, tool calling, usage fields, adapter status, and remaining contract gaps.

## Model Catalog

`/v1/gateway/model-catalog` explains what each public model name means. It shows the upstream model, provider status, fallback chain, capabilities, pricing metadata, request count, errors, tokens, and estimated cost. This helps customers understand that smart-fast can be a simple public name while the gateway manages the real provider route behind it.

## Model Route Lifecycle

The prototype can create, update, and disable public model routes. The dashboard has a Model route lifecycle panel for these demos. A route defines the customer-facing model name, provider, upstream model, fallback models, capabilities, and pricing metadata. These admin actions update `model_registry.json`, reload the local runtime, and write audit events. Production should add approval workflow, version history, rollback, and staged rollout.

## Route Preview

`/v1/gateway/route-preview` is a dry run. It does not call the provider. It shows whether a customer can use a model, whether budget is available, which model is primary, which models are fallback routes, which provider would handle the request, and whether that provider is ready.

## Route Decision Summary

Route preview and chat responses include `route_decision`. It explains the requested public model, selected upstream model, provider, fallback policy, provider controls, capability controls, and simple reasons for the routing decision. This is useful when a customer asks why a request used a certain model or provider.

## Safety Preview

`/v1/gateway/safety-preview` checks for obvious sensitive data before a provider call. It can detect examples such as email addresses, possible phone numbers, possible API keys, and secret assignments. Chat requests can also use `gateway_safety_check` to return safety metadata, `gateway_block_sensitive` to block detected sensitive data, or `gateway_redact_sensitive` to replace detected sensitive data before the provider call. This is useful for explaining gateway guardrails, but it is not a full DLP or compliance system.

## Provider Routing Control

The prototype supports `gateway_allowed_providers`. This lets a request say: only use these providers for this route. For example, route preview can allow only dashscope. If no route matches the allowed provider list, the gateway returns a clear error. This helps customers understand that the gateway is a control layer, not only a normal API proxy.

## Routing Strategy

The prototype supports `gateway_route_strategy`. Supported values are `registry`, `lowest_cost`, `fastest`, and `healthiest`. The strategy reorders route candidates before the provider call. Route preview and chat responses include `candidate_scores` in `route_decision`. This helps explain cost-aware, latency-aware, and health-aware routing. Production should use stronger cost data, latency windows, provider SLAs, and customer policy rules.



## Policy Presets

The prototype supports `gateway_policy`. A policy preset is a short name for common routing controls. Current presets include `balanced`, `lowest_cost`, `fastest`, and `tool_ready`. `/v1/gateway/policy-presets` lists the presets. Explicit request controls override preset controls. This helps non-technical customers choose a simple policy name instead of many technical fields.

## Customer Default Policy

A customer can have `default_policy` in `customer_keys.json`. If a request does not send `gateway_policy`, the gateway uses the customer default. If a request sends `gateway_policy`, the request value wins. This lets a customer use one gateway key and one default routing behavior without sending routing controls every time.

## Capability Routing Control

The prototype supports `gateway_required_capabilities`. This lets route preview or a real request require abilities such as streaming or tools. `stream=true` requires streaming. A tools request requires tools. If no route supports the required ability, the gateway returns a `clear no_capability_route` error. This explains why model metadata matters in a real gateway.

## Cost Estimate

`/v1/gateway/cost-estimate` is a dry run. It does not call the provider. It estimates prompt tokens, completion tokens, estimated cost, and remaining budget after the estimate. This is useful for planning, but real provider token usage can differ.

## Customer Key Issue Preview

`/v1/gateway/key-issue-preview` creates a safe customer onboarding package. It returns a generated gateway API key, a masked key for display, a customer config snippet, allowed models, limits, and next steps. It does not save the customer. In production, this would connect to a real customer database and secret manager.

## Customer Key Lifecycle

The prototype can also create a customer, rotate a customer gateway key, and disable a customer. The dashboard has a Customer lifecycle panel for these demos. These admin actions update `customer_keys.json` and reload the local runtime. After rotation, the old key stops working. After disable, the customer key stops working. This is useful for demos, but production should use a database, audit logs, approval workflow, and a secret manager.

## Audit Events

`/v1/gateway/audit-events` records admin lifecycle actions such as customer created, key rotated, customer disabled, provider changed, and model route changed. The dashboard also has an Audit timeline section for recent changes. Each event includes actor, action, target customer, provider, or model route, time, and safe details such as masked keys and key hashes. It does not store full gateway API keys. This helps explain control history, but it is not a full compliance audit system.

## Invoice Preview

`/v1/gateway/invoice-preview` uses local usage records to create an estimated billing preview. It shows requests, errors, prompt tokens, completion tokens, total tokens, estimated cost, remaining budget, usage by model, and usage by provider. It can return JSON or CSV. It is not a legal invoice, but it helps explain how customer billing

reports could work later.

## Customer Reports

`/v1/gateway/customer-reports` shows one report per customer. It includes request count, error count, token usage, remaining budget, models used, providers used, and recent requests. This helps explain that a gateway is also a control and reporting layer, not only a model router.

## Customer Success Summary

`/v1/gateway/customer-success` turns customer usage data into account health. It shows healthy, watch, and at-risk customer counts, health status per customer, risk reasons, recommended follow-up action, simple business metrics, meeting questions, and evidence endpoints for support. This helps sales, support, and customer success teams decide who needs attention before the next customer call.

## Request Activity

`/v1/gateway/request-activity` shows recent gateway decisions. It can filter by customer, model, provider, error code, status, and limit. This helps explain which customer sent a request, which public model was requested, which upstream model was used, which provider handled it, and whether the request succeeded or failed.

## Request Detail

Every successful chat response includes `gateway.request_id`. `/v1/gateway/request-detail` can look up one request by that id. It shows the customer, public model, upstream model, provider, outcome, latency, and nearby usage record when available.

## Config Check

The prototype includes `/v1/gateway/config-check`. It warns about demo admin keys, demo customer keys, missing live provider keys, and direct secret values in local JSON files. This is not a full security audit, but it is a useful readiness checklist for customer demos.

## Production Readiness Report

`/v1/gateway/production-readiness` explains go-live gaps in plain English. The dashboard also shows this as Production readiness cards. It groups the work into security, provider readiness, customer controls, routing and fallback, observability, billing, and documentation handoff. Each area has a status, evidence, and next step. This helps a customer understand why a prototype can be demo-ready but still not production-ready.

## Production Launch Plan

`/v1/gateway/launch-plan` converts readiness gaps into go-live gates. It explains security and secrets, provider live readiness, customer access and budgets, routing and fallback, support and observability, billing and commercial rules, and customer handoff. Each gate has an owner, required evidence, current evidence, approval question, status, and next step. It also shows required signoffs and rollout stages from internal live test to broader rollout.

## Change Management Plan

`/v1/gateway/change-management` explains how to change customers, providers, or model routes without surprising a customer. It includes change types, risk level, approval owner, before-change checklist, after-change validation, rollback path, and evidence endpoints. It is not automatic rollback yet, but it is a simple operating plan for safer

demos, pilots, and production discussions.

Recommended Roadmap

| Phase                | Goal                                                                               | Result                                  |
|----------------------|------------------------------------------------------------------------------------|-----------------------------------------|
| 1. Current prototype | Qwen-only gateway with mock dashboard                                              | Customer can understand the concept     |
| 2. Live Qwen demo    | Use Model Studio API key for real chat                                             | Validate latency and response quality   |
| 3. Gateway controls  | Persist logs, usage, model access rules, budgets, and BYOK for customer management | Enterprise-ready platform path          |
| 4. More providers    | Enable OpenAI, Claude, Xiaomi adapters                                             | More complete OpenRouter-like direction |
| 5. Production        | Database, secrets, streaming, fallback, billing                                    | Enterprise-ready platform path          |

Reference Documents

OpenRouter API Reference: <https://openrouter.ai/docs/api/reference/overview/>

OpenRouter Authentication: <https://openrouter.ai/docs/api-keys>

OpenRouter Limits: <https://openrouter.ai/docs/api-reference/limits/>

OpenRouter Models API: <https://openrouter.ai/docs/api/api-reference/models/get-models>

OpenRouter Model Fallbacks: <https://openrouter.ai/docs/guides/routing/model-fallbacks>

OpenRouter Provider Routing: <https://openrouter.ai/docs/guides/routing/provider-selection/>

OpenRouter BYOK: <https://openrouter.ai/docs/use-cases/byok/>

Alibaba Cloud Model Studio DashScope API Reference: <https://www.alibabacloud.com/help/doc-detail/3016809.html>

Final Recommendation

Start small and explain clearly. Use the visual dashboard to show the customer-facing API, the routing layer, and the provider adapter. Use mock mode for discussion and live Qwen mode only when real model testing is needed. Add more providers only after the customer understands the value of the gateway.